foreword

No one can argue that AJAX was an important advancement in the evolution of the web. In a few short years, a single technology (`XMLHttpRequest`) revolutionized how users interacted with our content. Instead of loading entire pages, portions of the page could refresh with minimal distraction to the user. In a time when broadband wasn't the norm, this change was amazingly powerful.

The web grew up during that time. The birth of AJAX catalyzed the transformation of "web pages" into "web apps," but it also paved the way for modern client-side development. Today's JavaScript frameworks, which launched single page apps (SPAs), were a result of this early paradigm shift. But as more code moved off the server and into the client, it was clear `XMLHttpRequest` wasn't keeping up. JavaScript's single-origin policy suffocated our creative potential. Web developers like you and I developed clever techniques (JSONP and proxy servers) to wiggle around the restrictions, but ultimately, all our cleverness was just a bandage. Gone were the days of the mashup. Web services were becoming a ubiquitous "back end" for web applications. True dependencies in our applications are critical to making web services tick. However, for services to be accessible from JavaScript meant a better tool was needed for dealing with remote resources. Enter cross-origin resource sharing, better known as CORs.

CORs is a powerful addition in the evolution of `XMLHttpRequest` and the advancement of web apps. By definition, CORs creates a standard way for JavaScript to securely communicate with cross-domain resources. Practically speaking, it opens up a whole new world for front-end developers. CORs brings back flexibility to JavaScript developers and allows them to access APIs and services from anywhere on web. For example,